

Accelerating Physical Design Jobs with tmpfs and rsync

A design and operations white paper for the PD Job Acceleration project

Authors: Project notes distilled from implementation and experiments in Physical-Design-Algorithms-Implementation

Scope: Job-local I/O acceleration for ASIC/SoC physical-design workloads on Linux compute farms

Companion code: PD Job Acceleration/ (orchestrators, demos, benchmarks)

Abstract

Physical-design (PD) tools—placement, CTS, routing, parasitic extraction, metal fill, STA, and ECO loops—are frequently described as CPU- or memory-bound. On industrial compute farms, however, a large fraction of wall-clock time is spent waiting on the **filesystem**, especially when the hot working directory lives on **NFS**. The dominant cost is often not raw sequential bandwidth but **per-operation latency** on huge numbers of tiny reads and writes (liberty-like lookups, SPEF shards, reports, tempfiles, incremental saves).

This white paper presents a practical, license-preserving acceleration pattern:

1. Treat storage as a **tiered system** (hot RAM vs durable disk/NFS).
2. Place only the **hot, chatty, sized-to-fit** working set on **tmpfs** (/dev/shm or an explicit tmpfs mount).
3. Keep **fat logs (often 10-50 GB+)** and other oversized artifacts on **local SSD or NFS**—never in RAM by default.
4. Use **rsync** as the durability bus: stage in, checkpoint incrementally, finalize on EXIT/TERM, then free RAM.

We describe two operating modes (full-workspace staging vs hybrid scratch), the safety model, implementation details, and measured results ranging from modest local-SSD gains to **~45-50×** speedups under an NFS-RTT farm model. The central claim is not a new PD algorithm; it is that **storage placement is part of turnaround**, and that this can be defended rigorously in design reviews and interviews.

Table of contents

1. [Motivation and industry context](#)
2. [Problem statement](#)
3. [Why page cache is not enough](#)
4. [Design principles](#)

5. [Architecture: two modes](#)
 6. [tmpfs as the hot tier](#)
 7. [rsync as the durability bus](#) 7a. [After rsync: shell sync for writeback durability](#) 7b. [perf: measure I/O vs CPU before you move data](#)
 8. [Log policy: why fat logs must stay off RAM](#)
 9. [Safety, failure modes, and operational rules](#)
 10. [Implementation in this repository](#)
 11. [Experimental methodology](#)
 12. [Results](#)
 13. [When the pattern helps—and when it does not](#)
 14. [Defending the approach \(review / interview\)](#)
 15. [Limitations and future work](#)
 16. [Conclusion](#)
 17. [Appendix A: quick-start commands](#)
 18. [Appendix B: glossary](#)
-

1. Motivation and industry context

1.1 The PD farm reality

A modern PD block run typically involves:

- reading large **LEF/DEF/Oasis/GDS** and timing libraries,
- maintaining a growing **design database**,
- emitting **reports**, logs, and intermediate saves,
- and iterating through **ECO** cycles under schedule pressure.

On a laptop with local NVMe, many of these paths are “fast enough.” On a shared farm, the same Tcl often runs with \$cwd on **NFS**. Every tiny open/stat/read/write can pay a network round-trip. CPUs look busy in the scheduler while spending time in **iowait**; licenses stay checked out; turnaround suffers.

1.2 Misdiagnosis is common

Teams often respond with:

- more cores / higher CPU priority,
- larger machines,
- or algorithm-level tuning,

when the binding constraint is **I/O placement**. Conversely, staging an entire multi-tens-of-GB workspace into RAM “because tmpfs is fast” can **OOM the node** or thrash shared /dev/shm.

This project exists to make the correct middle path explicit, measurable, and operationally safe.

2. Problem statement

Given a PD tool command and a durable job directory on disk/NFS,
accelerate wall-clock turnaround by relocating the hot working set onto RAM-backed storage,
without changing the tool binary, licenses, or functional results,
while preserving durability of required outputs across kills and avoiding RAM exhaustion from oversized artifacts (especially logs).

Success criteria:

1. **Functional equivalence** — deliverables match (checksums / reports).
 2. **Durability** — required outputs survive process kill via finalize/checkpoint.
 3. **Bounded RAM** — hot tier sized; fat logs excluded by default.
 4. **Operability** — one orchestrator command; clear Mode A vs Mode B choice.
-

3. Why page cache is not enough

Linux page cache already accelerates repeated reads of hot pages. It does **not** fully solve PD farm I/O for several reasons:

1. **Metadata RTT on NFS** — opening thousands of distinct small files still costs network round-trips even when data eventually caches.
2. **Writeback semantics** — durable commits, `fsync`, and NFS write stability behave differently from local tmpfs.
3. **Working-set churn** — ECO loops and report spam create many short-lived files that defeat “read it once, keep it hot” assumptions.
4. **Lack of policy** — page cache is global and opportunistic; PD jobs need an explicit **job-local scratch namespace** with a durability contract.

tmpfs + rsync is therefore not “reinventing cache”; it is **deliberate tiering with an explicit sync protocol**.

4. Design principles

1. **Tier by access pattern, not by file type slogan.**
Hot = high IOPS / many tiny ops / regenerable. Cold = huge / durable / final.
2. **Never make tmpfs the source of truth.**
Reboot or node death loses `/dev/shm`. Checkpoint or finalize always.
3. **Fat logs are a special case.**
Innovus/ICC2/FC-style logs routinely reach **10-50 GB+**. Putting them in RAM is a reliability bug disguised as optimization.

4. Measure boundness.

CPU-bound kernels (heavy fill/opt) gain little from I/O tiering. I/O-bound chatty phases gain a lot—especially under NFS-like latency.

5. Exclude regenerable junk from sync.

Shipping vault caches and temp DBs home can erase end-to-end wins.

6. Fail safe on signals.

Trap EXIT/INT/TERM so killed jobs still flush required outputs.

5. Architecture: two modes

Mode B (limited RAM – practical default)

disk / NFS		RAM (/dev/shm scratch)	
+-----+		+-----+	
design DB, libs		tmp/ (symlink)	
scripts		TMPDIR	
logs/ (STAY HERE)		small regenerable	
final outputs	flush	scratch only	
+-----+		+-----+	

Mode A (only if the working tree fits)

durable	--rsync (excl. logs)-->	tmpfs workspace
	<--rsync checkpoints--	tool cwd here
logs/	rewired to durable disk even in Mode A	

5.1 Mode B — hybrid scratch (ram_scratch.sh)

Keep the job directory on durable storage. Redirect only selected relative paths (default: tmp) into /dev/shm/... via symlinks, and export TMPDIR/TMP/TEMP into that scratch. On teardown, materialize needed paths (or discard regenerable scratch) and free RAM.

Use when: RAM is limited; design DB is large; logs are huge.

5.2 Mode A — full workspace (run_pd_job.sh)

Stage the durable tree into a RAM workspace, run the tool there, checkpoint with rsync, finalize, cleanup.

Critical default: PD_KEEP_LOGS_ON_DISK=1 rewires workspace logs/ to the durable logs directory and excludes logs from rsync (pattern logs, not only logs/, so a symlink named logs cannot overwrite the durable directory—a real bug we hit and fixed).

Use when: design + scratch fit in free RAM with headroom.

6. tmpfs as the hot tier

6.1 What tmpfs is

tmpfs is a filesystem backed by **pageable kernel memory** (and swap, if configured). On most Linux hosts, /dev/shm is already a tmpfs mount shared across jobs. Dedicated mounts (`mount -t tmpfs -o size=64G ...`) are useful for hard caps when privileges allow.

6.2 Properties that matter for PD

Property	Implication
Low latency	Tiny random ops avoid NFS RTT
High bandwidth	Checkpoint/DB stream writes are cheap
Volatile	Must checkpoint
Capacity = RAM policy	Size for peak, not input-only
Often noexec on /dev/shm	Run tool binaries from durable disk

6.3 noexec caveat

Some environments mount /dev/shm with noexec. Mode A must invoke ELF tools from a durable path (absolute path to .../scripts/run_fill, rcx_extract, etc.) even when data cwd is on tmpfs. Our RC Extraction and metal-fill accelerated runners do this explicitly.

7. rsync as the durability bus

7.1 Why rsync instead of `cp -a`

- **Incremental checkpoints** after the first full sync
- **Excludes** for regenerable paths
- **--partial** tolerance on flaky NFS
- Easy to run on a **timer** while the tool executes
- Familiar operations language for silicon CAD flows

7.2 Sync points

1. **Stage-in** (Mode A): durable → tmpfs
2. **Checkpoint loop** (optional): tmpfs → durable every (N) seconds
3. **Finalize**: last sync + STATUS + cleanup

7.3 Exclude policy

Default excludes include status dirs, editor junk, cores, and—when `PD_KEEP_LOGS_ON_DISK=1`—logs / logs/**. Workload-specific excludes (e.g. `tmp/lib_vault/`, `intermediate *.db`) prevent “winning on tool time, losing on shipping trash home.”

7a. After rsync: shell `sync` for writeback durability

rsync $\neq$ durable on media. rsync copies into the destination page cache. Until writeback is flushed, a node crash or abrupt NFS client death can erase a checkpoint that the orchestrator already logged as successful.

This toolkit therefore pairs rsync with the shell **sync** command (`pd_durable_sync` in `pd_job_env.sh`):

PD_SYNC_MODE	Behavior
off	No flush (fastest; weakest crash claim)
file	Write + sync a marker under <code>.pd_job_status/</code>
fs (default)	<code>sync -f</code> on the filesystem containing the durable root
global	Full-machine sync (avoid on busy farm nodes)

Defaults:

- `PD_SYNC_AFTER_FINALIZE=1` — flush after final rsync / Mode B materialize.
- `PD_SYNC_AFTER_CHECKPOINT=0` — per-checkpoint flush is optional; on NFS it can dominate wall time. Enable when the redo window matters more than farm load.

This is complementary to the application-level `fsync` some tools already issue on DB saves. The orchestrator cannot assume every EDA binary does that for every artifact you care about.

7b. perf: measure I/O vs CPU before you move data

tmpfs helps I/O-dominated phases. It does almost nothing for pure compute (timing graph walks, dense numerical kernels, many metal-fill CPU paths). Shipping a workspace into RAM without evidence wastes ops time and RAM quota.

`scripts/pd_perf_profile.sh` wraps a command with:

1. **perf stat** when a working perf binary exists (soft events always; HW cycles/IPC when the PMU is exposed),
2. **GNU time -v** for major page faults, voluntary context switches, and filesystem input/output counts,
3. a short **classification** (I/O-bound / CPU-bound / Mixed) and Mode A/B recommendation in `SUMMARY.txt`.

Enable in-line:

```
PD_PERF=1 PD_TOOL_CMD='...' ./scripts/ram_scratch.sh run
# or
./scripts/pd_perf_profile.sh --out logs/perf -- bash -c '...'
```

What perf is not: it is not I/O bandwidth isolation, and it is not a substitute for tmpfs. It is the measurement half of the loop:

profile → classify → place hot paths on tmpfs → rsync durability → sync writeback → re-profile.

Farm VMs often report <not supported> for HW counters; soft events plus `time -v` remain enough to catch “user+sys much less than wall” I/O wait patterns.

8. Log policy: why fat logs must stay off RAM

Commercial PD logs are append-heavy and frequently enormous. A 20 GB log in tmpfs:

- consumes RAM that the tool itself needs,
- risks ENOSPC on `/dev/shm` mid-route,
- and can OOM neighboring jobs on a shared node.

Policy adopted here:

- Default Mode B: `PD_RAM_PATHS=tmp` (**not** logs).
- Default Mode A: rewire `logs/` → durable disk.
- Prefer **local SSD/NVMe** for fat logs when available; NFS if necessary.
- Relative `-log logs/route.log` remains convenient because Mode A rewiring preserves the path while landing bytes on disk.

This policy is a first-class design constraint, not an afterthought.

9. Safety, failure modes, and operational rules

9.1 Failure modes

Failure	What happens	Mitigation
Tool killed (SIGTERM)	Trap runs finalize	Always trap EXIT/INT/TERM
Orchestrator killed -9	No finalize	Checkpoint loop reduces loss window
Host reboot	tmpfs gone	Only last successful sync survives
Undersized tmpfs	ENOSPC mid-job	Size for peak + headroom
Shared <code>/dev/shm</code> exhaustion	Multi-job interference	Per-user namespaces + quotas
rsync exclude bug (logs/ vs symlink logs)	Durable logs become self-symlink	Exclude logs and logs/***; repair if symlink

9.2 Operational rules (post these next to the farm wiki)

1. Never treat tmpfs as source of truth.

2. Do not put 20 GB+ logs in RAM.
 3. Size RAM for peak scratch/workspace.
 4. Exclude regenerable junk from sync.
 5. Trap signals; checkpoint long jobs.
 6. Namespace `/dev/shm/pdjobs/$USER/$JOB`.
 7. Measure whether the phase is CPU- or I/O-bound (`pd_perf_profile.sh`) before promising speedup.
 8. After finalize, flush writeback (`PD_SYNC_MODE=fs`) so durable media has the data.
-

10. Implementation in this repository

10.1 Core scripts

Script	Role
<code>scripts/pd_job_env.sh</code>	Shared defaults, rsync excludes, log rewiring, <code>pd_durable_sync</code>
<code>scripts/stage_to_tmpfs.sh</code>	Durable → tmpfs staging
<code>scripts/checkpoint_sync.sh</code>	Incremental hot → durable sync (+ optional sync)
<code>scripts/finalize_job.sh</code>	Final sync, durability sync, STATUS, cleanup
<code>scripts/run_pd_job.sh</code>	Mode A orchestrator (<code>PD_PERF=1</code> optional)
<code>scripts/ram_scratch.sh</code>	Mode B hybrid scratch
<code>scripts/pd_perf_profile.sh</code>	perf + GNU time I/O-vs-CPU classifier
<code>scripts/bench_io.sh</code>	Disk vs tmpfs microbench

10.2 Workloads used for evaluation

Workload	Nature	Path
Synthetic demos	Tiny I/O demos	<code>run_pd_job.sh --demo</code> , <code>ram_scratch.sh --demo</code>
RC Extraction	Mixed C++ extractor	<code>examples/run_rcx_accelerated.sh</code> , <code>compare_accel.sh</code>
<code>gpu_metal_fill</code>	CPU-heavy BEOL fill	<code>examples/compare_metal_fill.sh</code>
Local I/O-bound PD job	Liberty vault + checkpoints	<code>examples/compare_pd_io.sh</code>
Farm I/O suite	Multi-phase + NFS RTT model	<code>examples/compare_pd_farm_io.sh</code>

10.3 Notable implementation fixes

- **Paths with spaces** — tool commands run via `bash -c`, not unquoted `eval`.
- **EXIT trap locals** — orchestrator state used by traps must be global.
- **Log symlink vs rsync** — exclude logs not only `logs/`, otherwise `finalize` can replace a real durable directory with a self-referential symlink.

- `/dev/shm noexec` — execute binaries from durable storage.
-

11. Experimental methodology

11.1 Metrics

- **tool_s** — time from job start_epoch to done_epoch (compute + hot I/O).
- **e2e_s** — wall time including stage/rsync/teardown.
- **Functional checks** — checksums of deliverables (SPEF/GDS/lookup stamp).
- **Placement checks** — tmp_resolved / logs_resolved paths.

11.2 Fairness notes

1. **Local SSD vs tmpfs** measures a best-case disk; gains are often modest.
2. **Farm model** adds emulated per-op NFS RTT (e.g. 400 μ s) on the baseline path only, representing metadata latency common on networked home directories. Accelerated modes run identical work with `nfs-us=0` on tmpfs.
3. Regenerable scratch is not blindly flushed home in the farm/I/O compares (deliverables already copied to outputs/), so e2e is not dominated by shipping trash.

This methodology is intentionally honest: overselling local-SSD microbenches as “50 $\times$ ” would be indefensible in a design review.

12. Results

Results below were collected in the project’s Linux evaluation environment. Absolute times vary by host; ratios and qualitative conclusions are the point.

12.1 RC Extraction (mixed)

Same extract job with scratch-oriented I/O. On local fast disk, tool time improved on the order of **~15-20%** for accelerated modes in earlier compare runs; functional SPEF checksums matched. Useful as a “real binary” smoke test, not as the headline NFS claim.

12.2 gpu_metal_fill (CPU-bound control)

BEOL dummy fill on a synthetic GPU-block GDS (`SM_GRID=4`, two passes, ~137 MB filled GDS). Fill kernel runtime dominates (~6 s/pass). Accelerated modes showed only **~1%** tool-time improvement; filled GDS checksums were **identical**.

Interpretation: if the phase is CPU-bound, storage tiering will not create a miracle. Including this result strengthens credibility.

12.3 Local I/O-bound PD job

40k liberty-like cells, 400k random lookups, checkpoints, report spam on local disk:

config	tool_s	vs baseline	e2e_s	vs baseline
baseline (disk)	4.540	—	4.630	—
Mode B (tmp in RAM)	3.763	+17.1%	4.018	+13.2%
Mode A (workspace tmpfs)	3.339	+26.5%	3.691	+20.3%

Checksum identical across configs.

12.4 Farm I/O suite (headline result)

Multi-phase suite: liberty vault → random lookups → SPEF shards → ECO checkpoints → report spam. Baseline uses **400 μs emulated NFS RTT/op**.

config	tool_s	speedup	e2e_s	speedup
baseline (disk + NFS RTT)	114.890	—	114.955	—
Mode B (tmpfs, no RTT)	2.600	44.2×	2.880	39.9×
Mode A (tmpfs, no RTT)	2.279	50.4×	2.587	44.4×

Hottest phase example: liberty lookups **71.2 s → ~1.4 s**.

Functional checksum matched (35702796).

Interpretation: this is the regime the architecture targets—chatty PD I/O under networked filesystem latency.

12.5 Phase breakdown (farm suite, illustrative)

phase	baseline (NFS model)	Mode A (tmpfs)
liberty_vault	~19.1 s	~0.33 s
lib_lookups	~71.2 s	~1.45 s
spef_shards	~21.5 s	~0.41 s
eco_checkpoints	~0.09 s	~0.03 s
report_spam	~2.9 s	~0.05 s
all	~114.9 s	~2.3 s

13. When the pattern helps—and when it does not

Helps

- Hot directories on NFS or other high-RTT stores
- Many tiny files (libs, SPEF shards, reports, tempfiles)

- Iterative ECO loops reusing a hot tree
- TMPDIR currently pointing at network storage
- Tools that stream checkpoints frequently

Does not magically help

- Pure CPU kernels with little I/O (heavy fill/opt examples)
- Working sets larger than available RAM
- Jobs that already run entirely on local NVMe with warm cache
- Flows that write huge logs into the hot tree without redirection

Decision rule

```
if peak_hot_bytes << free_RAM and I/O_wait is visible:
    prefer Mode A (still keep fat logs on disk)
elif RAM limited or DB huge:
    Mode B: tmp/TMPDIR only (measure before adding paths)
else:
    fix log/output paths first; don't stage blindly
```

14. Defending the approach (review / interview)

Claim (precise)

Storage placement is part of PD turnaround. Relocate chatty, sized-to-fit working sets onto tmpfs; keep durability and fat logs on disk; sync with rsync under an explicit failure model. Speedup depends on whether the phase is I/O-bound under high-latency storage.

Strong answers to pushback

“Isn’t this just cache?”

Page cache is opportunistic and global. This is job-scoped tiering with a durability protocol and exclude policy.

“tmpfs will OOM us.”

Correct if misused. Defaults keep logs off RAM; Mode B only redirects tmp; size for peak; namespace and quota.

“What about node death?”

Volatile by design; checkpoints define redo budget. Same class of risk as any local scratch disk without sync.

“Show me it’s not fake.”

Publish both local-disk and NFS-model numbers; include a CPU-bound negative control (metal fill); require checksum equality.

“Why rsync?”

Incremental, excludable, partial-transfer tolerant, timer-friendly—boring on purpose.

15. Limitations and future work

1. **NFS model is an emulator** in the headline suite (fixed $\mu\text{s/op}$). Real NFS traces (e.g. bpftrace/strace histograms) would calibrate better.
 2. **No automatic working-set sizing yet** — operators must estimate peak.
 3. **Vendor tempfile paths** do not always honor TMPDIR; discovery via lsof/strace/perf is still partly manual.
 4. **Multi-tenant fair sharing** of /dev/shm needs cgroup integration (true I/O bandwidth isolation is io.max, not nice/ionice).
 5. **Integration with LSF/SGE/k8s** prologue/epilogue hooks is natural next packaging work.
 6. **Optional local SSD tier** between NFS and tmpfs for fat logs and medium scratch would complete a three-tier story.
 7. **Richer perf record/perf report flame graphs** for phase-level attribution (currently perf stat + classification; record is hinted).
-

16. Conclusion

Physical-design turnaround is a **system** problem: algorithms, licenses, CPUs, **and** storage. The tmpfs + rsync pattern gives a concrete, teachable control knob:

- hot chatty I/O → RAM,
- fat logs / finals → disk,
- durability → deliberate rsync + writeback sync,
- claims → measured with perf / GNU time against CPU-bound and I/O-bound controls.

Used carefully, it preserves correctness while removing a class of farm latency that no amount of Tcl reorganization can fix. Used carelessly (especially with huge logs in RAM), it becomes an outage generator. This repository encodes the careful version—modes, defaults, traps, excludes, and benchmarks—so the approach can be demonstrated, reviewed, and defended.

Appendix A: quick-start commands

```
cd "PD Job Acceleration"

# Demos (no EDA license)
./scripts/ram_scratch.sh --demo
./scripts/run_pd_job.sh --demo
./examples/demo_perf_and_sync.sh
```

```
# Real binaries
./examples/run_rcx_accelerated.sh
./examples/compare_metal_fill.sh

# I/O-bound compares
./examples/compare_pd_io.sh
./examples/compare_pd_farm_io.sh    # monumental NFS-model vs tmpfs

# Microbench
./scripts/bench_io.sh 256
```

Environment knobs (selected):

variable	meaning	default
PD_DURABLE_ROOT	durable job directory	required
PD_JOB_NAME	workspace name slice	timestamped
PD_RAM_PATHS	Mode B relative dirs in RAM	tmp
PD_KEEP_LOGS_ON_DISK	Mode A log rewiring	1
PD_CHECKPOINT_SECS	checkpoint period	0 (off)
PD_FLUSH_ON_TEARDOWN	Mode B materialize scratch	1
PD_EXCLUDE_FILE	extra rsync excludes	empty
PD_TOOL_CMD	command run in job cwd	required
PD_SYNC_MODE	writeback flush mode after rsync	fs
PD_SYNC_AFTER_FINALIZE	shell sync after finalize/flush	1
PD_SYNC_AFTER_CHECKPOINT	shell sync after each checkpoint	0
PD_PERF	wrap tool with pd_perf_profile.sh	0

Appendix B: glossary

term	meaning
tmpfs	RAM-backed Linux filesystem
/dev/shm	common shared tmpfs mount
Mode A	full workspace staged into tmpfs
Mode B	hybrid: only scratch paths in RAM
durable	NFS/disk source of truth
checkpoint	incremental rsync hot → durable
finalize	last sync + status + cleanup
sync(1)	flush kernel writeback to durable media (≠ rsync)
perf	Linux performance counters / profiling toolkit

term	meaning
NFS RTT	network round-trip time paid per remote op
ECO	engineering change order iteration
SPEF	Standard Parasitic Exchange Format
liberty	.lib timing/power cell models

Document history

version	notes
1.0	Initial white paper aligned with PD Job Acceleration implementation, Mode A/B defaults, log policy, and farm I/O suite results
1.1	Added perf profiling loop and shell sync durability after rsync